

Министерство образования Республики Беларусь  
Учреждение образования  
«Брестский государственный технический университет»  
Кафедра ИИТ

**Лабораторная работа №2**

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для задачи «исключающее ИЛИ» с числовыми классами:  
сравнение функций потерь MSE и Binary Cross-Entropy»

**Выполнил:**

Студент 4 курса

Группы ИИ-26

Суховило В. С.

**Проверил:**

Никитин С. К.

**Цель работы:** На основе многослойного персептрона архитектуры 2-2-1, реализованного в Лабораторной работе №1 для задачи XOR с числовыми классами  $s_0$  и  $s_1$  из диапазона  $[-10; 10]$ , реализовать альтернативную функцию потерь – бинарную кросс-энтропию (Binary Cross-Entropy, BCE) – и провести сравнительный анализ обучения сети с MSE (Лабораторная работа №1) и с BCE по скорости сходимости, качеству обучения и устойчивости к выбору начальных весов.

**Ход работы:**

Смысл лабораторной №2 — взять многослойный персептрон **2–2–1** из ЛР №1 и сравнить две конфигурации:

**А — MSE**

исходные цели  $-9$  и  $-8$ ;

сигмоида;

обучение как в ЛР №1.

**В — BCE**

классы надо преобразовать:

$-9 \rightarrow 0$

$-8 \rightarrow 1$ ;

обучать сеть через **Binary Cross-Entropy**;

затем выход  $\hat{y}$  из диапазона  $(0,1)$  преобразовывать обратно в шкалу  $[-9; -8]$ . PSAI 2.2

Например:

$\hat{y} = 0 \rightarrow -9$

$\hat{y} = 1 \rightarrow -8$

$\hat{y} = 0.72 \rightarrow -8.28$

Также по методичке нужно будет сделать **минимум 5 запусков для MSE и 5 для BCE с разными seed**, сравнить количество эпох, ошибку, ассурасу и MAE. PSAI 2.2

Из графиков понадобятся:

MSE и BCE — ошибка от эпохи на одном графике;

сравнение числа эпох для 5 seed;

разделяющая поверхность для обеих сетей;

сравнение MAE. PSAI 2.2

И ещё должен быть **режим функционирования сети**, куда вводим любые  $A, B \in [-10; 10]$ , а программа показывает нормализованный выход, результат в исходной шкале и ближайший класс  $-9$  или  $-8$ . Нужно проверить 4 обучающих примера и ещё 2–3 произвольных. PSAI 2.2

Важный момент: **нельзя использовать PyTorch, TensorFlow, scikit-learn**. Разрешены NumPy и Matplotlib, а обратное распространение надо реализовать самостоятельно.

### Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

# =====
# Лабораторная работа №2
# Вариант 3: c0 = -9, c1 = -8
# Многослойный персептрон 2-2-1
# Сравнение MSE и Binary Cross-Entropy
#
# ВАЖНО:
# Конфигурация А (MSE) сравнивает выход сети В ИСХОДНОЙ
# шкале [-9; -8] с исходными целями -9 и -8.
#
# Конфигурация В (BCE) обучается на бинарных целях 0/1,
# как требуется в методичке.
#
# Сигмоида физически выдаёт только (0; 1), поэтому её выход
# переводится в исходную шкалу формулой:
# y_real = c0 + y_norm * (c1 - c0)
#
# Для варианта 3:
# y_real = -9 + y_norm
# =====

# -----
# 1. Исходные данные варианта 3
# -----

C0 = -9.0
C1 = -8.0

X_REAL = np.array([
    [-9.0, -9.0],
    [-9.0, -8.0],
    [-8.0, -9.0],
    [-8.0, -8.0]
], dtype=float)

Y_REAL = np.array([
    [-9.0],
    [-8.0],
    [-8.0],
    [-9.0]
], dtype=float)

# Гиперпараметры
LEARNING_RATE = 0.5

# Пороги различаются, потому что MSE и BCE имеют разный масштаб.
EE_MSE = 0.01
EE_BCE = 0.03

MAX_EPOCHS = 20000

# Не менее 5 различных случайных инициализаций
SEEDS = [7, 8, 9, 10, 11]

# Одинаковый порядок перемешивания примеров для обеих конфигураций
SHUFFLE_SEED = 42
```

```

# -----
# 2. Масштабирование
# -----

def to_binary_scale(value, c0=C0, c1=C1):
    """
    Перевод исходной числовой шкалы [c0; c1] в [0; 1].

    Для варианта 3:
    -9 -> 0
    -8 -> 1
    """
    return (value - c0) / (c1 - c0)

def to_real_scale(y_norm, c0=C0, c1=C1):
    """
    Обратный перевод нормализованного выхода в исходную шкалу.

    y_real = c0 + y_norm * (c1 - c0)

    Для варианта 3:
    y_real = -9 + y_norm
    """
    return c0 + y_norm * (c1 - c0)

# Входы масштабируем одинаково для обеих конфигураций.
# Это только численная предобработка и не меняет классы XOR.
#
# Для обучающей таблицы:
# -9 -> 0
# -8 -> 1
X = to_binary_scale(X_REAL)

# Бинарные цели нужны для BCE и для вычисления ассурасу.
Y_BINARY = to_binary_scale(Y_REAL)

```

```

# -----
# 3. Вспомогательные функции
# -----

```

```

def sigmoid(z):
    z = np.clip(z, -60.0, 60.0)
    return 1.0 / (1.0 + np.exp(-z))

def sigmoid_derivative_from_output(y):
    return y * (1.0 - y)

def mse_real_loss(y_true_real, y_pred_norm):
    """
    Конфигурация A.

    MSE считается В ИСХОДНОЙ ШКАЛЕ:
    сначала y_norm -> y_real,
    затем сравнение с исходными целями -9/-8.

    Es = 1/2 * sum((y_real - y_true_real)^2)
    """
    y_pred_real = to_real_scale(y_pred_norm)

    return 0.5 * np.sum(
        (y_pred_real - y_true_real) ** 2
    )

```

```

def bce_loss(y_true_binary, y_pred_norm):
    """
    Конфигурация В.

    BCE считается по нормализованным целям 0/1.
    """
    eps = 1e-12
    y_pred_norm = np.clip(
        y_pred_norm,
        eps,
        1.0 - eps
    )

    return -np.sum(
        y_true_binary * np.log(y_pred_norm)
        + (1.0 - y_true_binary) * np.log(1.0 - y_pred_norm)
    )


def nearest_class_from_real(y_real):
    """
    Определяет, к какому исходному классу значение ближе.
    """
    d0 = np.abs(y_real - C0)
    d1 = np.abs(y_real - C1)

    return np.where(d0 <= d1, C0, C1)


def classification_accuracy(y_true_real, y_pred_norm):
    """
    Аккуратность считается в исходных классах c0/c1.
    """
    y_pred_real = to_real_scale(y_pred_norm)

    true_classes = nearest_class_from_real(y_true_real)
    pred_classes = nearest_class_from_real(y_pred_real)

    return np.mean(true_classes == pred_classes)


def mean_absolute_error_real(y_true_real, y_pred_norm):
    """
    Средняя абсолютная ошибка в исходной шкале [c0; c1].
    """
    y_pred_real = to_real_scale(y_pred_norm)

    return np.mean(
        np.abs(y_true_real - y_pred_real)
    )


# -----
# 4. MLP 2-2-1
# -----

class MLP221:
    def __init__(self, seed=7):
        rng = np.random.RandomState(seed)

        # Малые случайные веса
        self.W1 = rng.uniform(
            -0.1, 0.1,
            size=(2, 2)
        )

        self.b1 = rng.uniform(

```

```

        -0.1, 0.1,
        size=(1, 2)
    )

    self.W2 = rng.uniform(
        -0.1, 0.1,
        size=(2, 1)
    )

    self.b2 = rng.uniform(
        -0.1, 0.1,
        size=(1, 1)
    )

def forward(self, x):
    """
    Прямое распространение.
    Сигмоида используется на скрытом и выходном слоях.
    """

    net1 = x @ self.W1 + self.b1
    hidden = sigmoid(net1)

    net2 = hidden @ self.W2 + self.b2
    output_norm = sigmoid(net2)

    return hidden, output_norm

def predict_norm(self, x):
    return self.forward(x)[1]

def train(
    self,
    x_train,
    y_real_train,
    y_binary_train,
    loss_name,
    learning_rate,
    ee,
    max_epochs,
    shuffle_seed
):
    """
    Онлайн-обучение:
    обновление весов после каждого примера.
    """

    rng_shuffle = np.random.RandomState(
        shuffle_seed
    )

    history = []
    converged = False

    for epoch in range(1, max_epochs + 1):

        indices = np.arange(
            len(x_train)
        )
        rng_shuffle.shuffle(indices)

        for i in indices:

            x = x_train[i:i + 1]
            target_real = y_real_train[i:i + 1]
            target_binary = y_binary_train[i:i + 1]

            # -----

```

```

# Прямое распространение
# -----
hidden, output_norm = self.forward(x)

# -----
# Выходной градиент
# -----

if loss_name == "mse":

    # Сначала переводим нормализованный выход
    # в исходную шкалу.
    output_real = to_real_scale(
        output_norm
    )

    #  $E = 1/2 * (y_{\text{real}} - \text{target\_real})^2$ 
    #
    #  $y_{\text{real}} =$ 
    #  $c_0 + \text{output\_norm} * (c_1 - c_0)$ 
    #
    #  $dE/dnet =$ 
    #  $(y_{\text{real}} - \text{target\_real})$ 
    #  $* (c_1 - c_0)$ 
    #  $* \text{sigmoid}'(\text{net})$ 

    delta2 = (
        (output_real - target_real)
        * (C1 - C0)
        * sigmoid_derivative_from_output(
            output_norm
        )
    )

elif loss_name == "bce":

    # Для BCE + sigmoid:
    #
    #  $dE/dnet = y_{\text{hat}} - y$ 
    #
    # Цель здесь бинарная: 0 или 1.

    delta2 = (
        output_norm
        - target_binary
    )

else:
    raise ValueError(
        "loss_name должен быть 'mse' или 'bce'"
    )

# -----
# Градиент скрытого слоя
# -----

delta1 = (
    (delta2 @ self.W2.T)
    * sigmoid_derivative_from_output(
        hidden
    )
)

# -----
# Онлайн-обновление весов
# -----

self.W2 -= (

```

```

        learning_rate
        * hidden.T
        @ delta2
    )

    self.b2 -= (
        learning_rate
        * delta2
    )

    self.W1 -= (
        learning_rate
        * x.T
        @ delta1
    )

    self.b1 -= (
        learning_rate
        * delta1
    )

# -----
# Суммарная ошибка после полной эпохи
# -----

predictions_norm = self.predict_norm(
    x_train
)

if loss_name == "mse":
    error = mse_real_loss(
        y_real_train,
        predictions_norm
    )

else:
    error = bce_loss(
        y_binary_train,
        predictions_norm
    )

history.append(
    float(error)
)

if error <= ee:
    converged = True
    break

return {
    "epochs": epoch,
    "final_error": float(error),
    "converged": converged,
    "history": history
}

```

```

# -----
# 5. Один запуск
# -----

```

```
def run_one_configuration(loss_name, seed):
```

```
    model = MLP221(seed=seed)
```

```
    if loss_name == "mse":
```

```
        ee = EE_MSE
```

```
    else:
```

```

ee = EE_BCE

training = model.train(
    x_train=X,
    y_real_train=Y_REAL,
    y_binary_train=Y_BINARY,
    loss_name=loss_name,
    learning_rate=LEARNING_RATE,
    ee=ee,
    max_epochs=MAX_EPOCHS,
    shuffle_seed=SHUFFLE_SEED
)

```

```

pred_norm = model.predict_norm(X)
pred_real = to_real_scale(pred_norm)

```

```

accuracy = classification_accuracy(
    Y_REAL,
    pred_norm
)

```

```

mae = mean_absolute_error_real(
    Y_REAL,
    pred_norm
)

```

```

return {
    "loss_name": loss_name,
    "seed": seed,
    "model": model,

    "epochs": training["epochs"],
    "final_error": training["final_error"],
    "converged": training["converged"],
    "history": training["history"],

    "pred_norm": pred_norm,
    "pred_real": pred_real,

    "accuracy": float(accuracy),
    "mae": float(mae)
}

```

```

# -----
# 6. Пять запусков
# -----

```

```

def run_experiments():

```

```

    mse_runs = []
    bce_runs = []

    print("\n" + "=" * 78)
    print("ЛАБОРАТОРНАЯ РАБОТА №2 — ВАРИАНТ 3")
    print("c0 = -9, c1 = -8")
    print("=" * 78)

    print(
        "\nКонфигурация A: "
        "MSE в исходной шкале [-9; -8]"
    )

```

```

for seed in SEEDS:

```

```

    result = run_one_configuration(
        "mse",
        seed
    )

```

```

    )

    mse_runs.append(result)

    status = (
        "ДА"
        if result["converged"]
        else "НЕТ"
    )

    print(
        f"seed={seed:2d} | "
        f"эпох={result['epochs']:5d} | "
        f"Es={result['final_error']:.6f} | "
        f"accuracy={result['accuracy']:.2f} | "
        f"MAE={result['mae']:.6f} | "
        f"сходимость={status}"
    )

    print(
        "\nКонфигурация В: "
        "ВСЕ по нормализованным целям 0/1"
    )

    for seed in SEEDS:

        result = run_one_configuration(
            "bce",
            seed
        )

        bce_runs.append(result)

        status = (
            "ДА"
            if result["converged"]
            else "НЕТ"
        )

        print(
            f"seed={seed:2d} | "
            f"эпох={result['epochs']:5d} | "
            f"Es={result['final_error']:.6f} | "
            f"accuracy={result['accuracy']:.2f} | "
            f"MAE={result['mae']:.6f} | "
            f"сходимость={status}"
        )

    return mse_runs, bce_runs

# -----
# 7. Представительный запуск
# -----

def choose_representative_run(runs):

    successful = [
        r for r in runs
        if r["converged"]
    ]

    if successful:

        successful_epochs = np.array(
            [
                r["epochs"]
                for r in successful
            ]
        )

```

```

    ]
)

median_epochs = np.median(
    successful_epochs
)

return min(
    successful,
    key=lambda r:
        abs(
            r["epochs"]
            - median_epochs
        )
)

# Если успешных нет —
# берём наименьшую итоговую ошибку.
return min(
    runs,
    key=lambda r:
        r["final_error"]
)

# -----
# 8. Таблица предсказаний
# -----

def print_prediction_table(title, run):

    print("\n" + title)
    print("-" * 88)

    print(
        f"{'A':>7}"
        f"{'B':>8}"
        f"{'Цель':>10}"
        f"{'ŷ norm':>14}"
        f"{'ŷ real':>14}"
        f"{'Класс':>12}"
    )

    print("-" * 88)

    for (
        x_real,
        target_real,
        pred_norm,
        pred_real
    ) in zip(
        X_REAL,
        Y_REAL.ravel(),
        run["pred_norm"].ravel(),
        run["pred_real"].ravel()
    ):

        predicted_class = (
            C0
            if abs(pred_real - C0)
            <= abs(pred_real - C1)
            else C1
        )

        print(
            f"{x_real[0]:7.1f}"
            f"{x_real[1]:8.1f}"
            f"{target_real:10.1f}"

```

```

        f"{pred_norm:14.4f}"
        f"{pred_real:14.4f}"
        f"{predicted_class:12.1f}"
    )

```

```

# -----
# 9. Сводка серии запусков
# -----

```

```
def print_summary(mse_runs, bce_runs):
```

```

    print("\n" + "=" * 78)
    print("СВОДКА ПО 5 ЗАПУСКАМ")
    print("=" * 78)

```

```

    for name, runs in [
        ("MSE", mse_runs),
        ("BCE", bce_runs)
    ]:

```

```

        successful = [
            r for r in runs
            if r["converged"]
        ]

```

```
        print(f"\n{name}:")
```

```

        print(
            f" Достигли критерия: "
            f"{len(successful)} из {len(runs)}"
        )

```

```
        if successful:
```

```

            epochs = np.array(
                [
                    r["epochs"]
                    for r in successful
                ]
            )

```

```

            print(
                f" Минимум эпох: "
                f"{epochs.min()}"
            )

```

```

            print(
                f" Максимум эпох: "
                f"{epochs.max()}"
            )

```

```

            print(
                f" Среднее эпох: "
                f"{epochs.mean():.2f}"
            )

```

```

            print(
                f" Разброс: "
                f"{epochs.max() - epochs.min()}"
            )

```

```

    mean_accuracy = np.mean(
        [
            r["accuracy"]
            for r in runs
        ]
    )

```

```

mean_mae = np.mean(
    [
        r["mae"]
        for r in runs
    ]
)

print(
    f" Средняя ассурасы: "
    f"{mean_accuracy:.4f}"
)

print(
    f" Средняя MAE:      "
    f"{mean_mae:.6f}"
)

```

```

# -----
# 10. График сходимости
# -----

```

```

def plot_convergence(mse_run, bce_run):

```

```

    plt.figure(
        figsize=(10, 6)
    )

    mse_epochs = np.arange(
        1,
        len(mse_run["history"]) + 1
    )

    bce_epochs = np.arange(
        1,
        len(bce_run["history"]) + 1
    )

    plt.plot(
        mse_epochs,
        mse_run["history"],
        label=(
            f"MSE, seed="
            f"{mse_run['seed']}"
        )
    )

    plt.plot(
        bce_epochs,
        bce_run["history"],
        label=(
            f"BCE, seed="
            f"{bce_run['seed']}"
        )
    )

    plt.axhline(
        EE_MSE,
        linestyle="--",
        label=f"Ee MSE = {EE_MSE}"
    )

    plt.axhline(
        EE_BCE,
        linestyle=":",
        label=f"Ee BCE = {EE_BCE}"
    )

```

```

# У функций разные масштабы,
# поэтому логарифмическая ось удобнее.
plt.yscale("log")

plt.xlabel("Номер эпохи")
plt.ylabel("Суммарная ошибка Es")

plt.title(
    "Сравнение сходимости MSE и BCE"
)

plt.grid(
    True,
    alpha=0.3
)

plt.legend()
plt.tight_layout()

# -----
# 11. Эпохи по seed
# -----

def plot_epochs_by_seed(
    mse_runs,
    bce_runs
):

    x = np.arange(
        len(SEEDS)
    )

    width = 0.36

    mse_epochs = [
        r["epochs"]
        for r in mse_runs
    ]

    bce_epochs = [
        r["epochs"]
        for r in bce_runs
    ]

    plt.figure(
        figsize=(10, 6)
    )

    plt.bar(
        x - width / 2,
        mse_epochs,
        width,
        label="MSE"
    )

    plt.bar(
        x + width / 2,
        bce_epochs,
        width,
        label="BCE"
    )

    # Неуспешные запуски помечаем крестиком.
    for i, run in enumerate(mse_runs):

        if not run["converged"]:

```

```

plt.scatter(
    x[i] - width / 2,
    run["epochs"],
    marker="x",
    s=100,
    linewidths=2
)

for i, run in enumerate(bce_runs):

    if not run["converged"]:

        plt.scatter(
            x[i] + width / 2,
            run["epochs"],
            marker="x",
            s=100,
            linewidths=2
        )

plt.xticks(
    x,
    [str(seed) for seed in SEEDS]
)

plt.xlabel("Seed")
plt.ylabel("Количество эпох")

plt.title(
    "Устойчивость сходимости "
    "по 5 случайным инициализациям"
)

plt.legend()

plt.grid(
    axis="y",
    alpha=0.3
)

plt.tight_layout()

```

```

# -----
# 12. Разделяющая поверхность
# -----

```

```
def plot_decision_surface(run, title):
```

```

    model = run["model"]

    a_values = np.linspace(
        -10.0,
        10.0,
        250
    )

    b_values = np.linspace(
        -10.0,
        10.0,
        250
    )

    aa, bb = np.meshgrid(
        a_values,
        b_values
    )

```

```

grid_real = np.column_stack(
    [
        aa.ravel(),
        bb.ravel()
    ]
)

# Та же предобработка,
# что использовалась при обучении.
grid_scaled = to_binary_scale(
    grid_real
)

z_norm = model.predict_norm(
    grid_scaled
).reshape(
    aa.shape
)

plt.figure(
    figsize=(8, 7)
)

contour = plt.contourf(
    aa,
    bb,
    z_norm,
    levels=np.linspace(
        0.0,
        1.0,
        21
    ),
    cmap="viridis"
)

plt.colorbar(
    contour,
    label=(
        "Нормализованный "
        "выход  $\hat{y}$ "
    )
)

# Граница классов
plt.contour(
    aa,
    bb,
    z_norm,
    levels=[0.5],
    linewidths=2
)

true_classes = (
    Y_BINARY
    .ravel()
    .astype(int)
)

plt.scatter(
    X_REAL[:, 0],
    X_REAL[:, 1],
    c=true_classes,
    cmap="coolwarm",
    edgecolors="black",
    s=120,
    label="Обучающие примеры"
)

```

```

plt.xlim(-10, 10)
plt.ylim(-10, 10)

plt.xlabel("A")
plt.ylabel("B")

plt.title(title)

plt.legend()

plt.grid(
    True,
    alpha=0.2
)

plt.tight_layout()

# -----
# 13. Сравнение MAE
# -----

def plot_mae_comparison(
    mse_run,
    bce_run
):

    names = [
        "MSE",
        "BCE"
    ]

    values = [
        mse_run["mae"],
        bce_run["mae"]
    ]

    plt.figure(
        figsize=(7, 5)
    )

    bars = plt.bar(
        names,
        values
    )

    plt.ylabel(
        "Средняя абсолютная ошибка"
    )

    plt.title(
        "MAE представительных запусков "
        "в исходной шкале [-9; -8]"
    )

    plt.grid(
        axis="y",
        alpha=0.3
    )

    for bar, value in zip(
        bars,
        values
    ):

        plt.text(
            bar.get_x()

```

```

        + bar.get_width() / 2,
        bar.get_height(),
        f'{value:.5f}',
        ha="center",
        va="bottom"
    )

```

```
plt.tight_layout()
```

```

# -----
# 14. Режим функционирования
# -----

```

```
def infer_one(model, a, b):
```

```

    x_real = np.array(
        [[a, b]],
        dtype=float
    )

```

```
    # Применяем то же масштабирование входа.
```

```

    x_scaled = to_binary_scale(
        x_real
    )

```

```

    y_norm = float(
        model.predict_norm(
            x_scaled
        )[0, 0]
    )

```

```

    y_real = float(
        to_real_scale(
            y_norm
        )
    )

```

```

    nearest_class = (
        C0
        if abs(y_real - C0)
        <= abs(y_real - C1)
        else C1
    )

```

```

    return (
        y_norm,
        y_real,
        nearest_class
    )

```

```
def demonstrate_functioning(model):
```

```

    print("\n" + "=" * 78)
    print("РЕЖИМ ФУНКЦИОНИРОВАНИЯ — ВСЕ")
    print("=" * 78)

```

```
    # 4 обучающих + 3 дополнительных примера
```

```

    examples = [
        (-9.0, -9.0),
        (-9.0, -8.0),
        (-8.0, -9.0),
        (-8.0, -8.0),
        (-8.5, -8.5),
        (-9.5, -8.2),
        (-7.5, -9.2)
    ]

```

for a, b in examples:

```
(
    y_norm,
    y_real,
    nearest
) = infer_one(
    model,
    a,
    b
)

print(
    f"A={a:6.2f}, "
    f"B={b:6.2f} | "
    f"ŷ={y_norm:.4f} | "
    f"y_real={y_real:.4f} | "
    f"ближайший класс="
    f"{nearest:.1f}"
)
```

def interactive\_mode(model):

```
print("\n" + "=" * 78)
print("ИНТЕРАКТИВНЫЙ РЕЖИМ")
print(
    "Введите A и B "
    "из диапазона [-10; 10]."
)
print("Для выхода введите q.")
print("=" * 78)
```

while True:

```
text = input("\nA = ").strip()
```

```
if text.lower() == "q":
    break
```

```
try:
```

```
    a = float(text)
    b = float(
        input("B = ").strip()
    )
```

except ValueError:

```
    print(
        "Ошибка: "
        "нужно ввести число."
    )
    continue
```

```
if not (
    -10.0 <= a <= 10.0
    and
    -10.0 <= b <= 10.0
):
```

```
    print(
        "Ошибка: "
        "A и B должны быть "
        "в диапазоне [-10; 10]."
    )
    continue
```

```

    (
        y_norm,
        y_real,
        nearest
    ) = infer_one(
        model,
        a,
        b
    )

    print(
        f"Нормализованный "
        f"выход  $\hat{y} = \{y\_norm:.4f\}$  "
    )

    print(
        f"Выход в исходной "
        f"шкале [{C0:.0f}; {C1:.0f}] "
        f"= {y_real:.4f}"
    )

    print(
        f"Ближайший класс "
        f"= {nearest:.1f}"
    )

# -----
# 15. Главная функция
# -----

def main():

    mse_runs, bce_runs = (
        run_experiments()
    )

    mse_rep = (
        choose_representative_run(
            mse_runs
        )
    )

    bce_rep = (
        choose_representative_run(
            bce_runs
        )
    )

    print_summary(
        mse_runs,
        bce_runs
    )

    print_prediction_table(
        "Представительный запуск MSE",
        mse_rep
    )

    print_prediction_table(
        "Представительный запуск BCE",
        bce_rep
    )

    demonstrate_functioning(
        bce_rep["model"]
    )

```

```

# Требуемые графики
plot_convergence(
    mse_rep,
    bce_rep
)

plot_epochs_by_seed(
    mse_runs,
    bce_runs
)

plot_decision_surface(
    mse_rep,
    (
        "Разделяющая поверхность "
        f"MSE, seed={mse_rep['seed']}"
    )
)

plot_decision_surface(
    bce_rep,
    (
        "Разделяющая поверхность "
        f"BCE, seed={bce_rep['seed']}"
    )
)

plot_mae_comparison(
    mse_rep,
    bce_rep
)

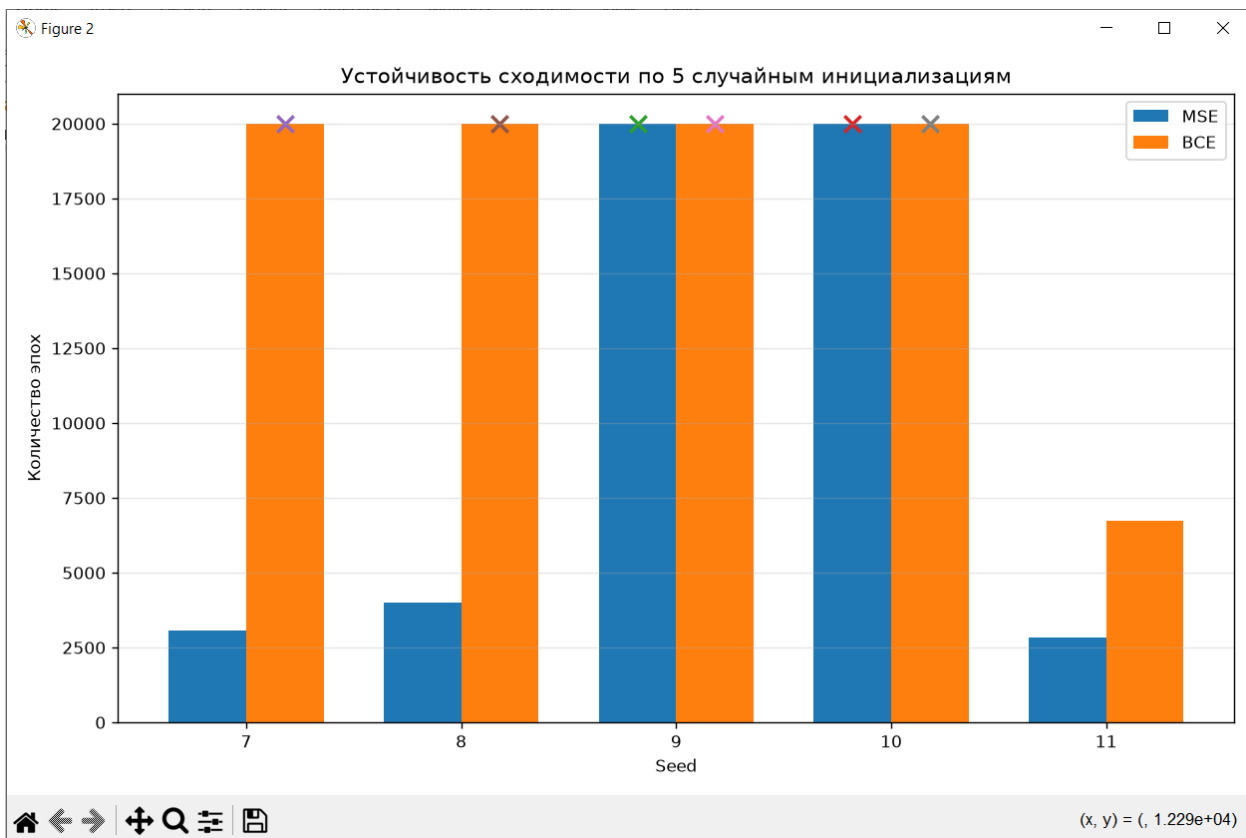
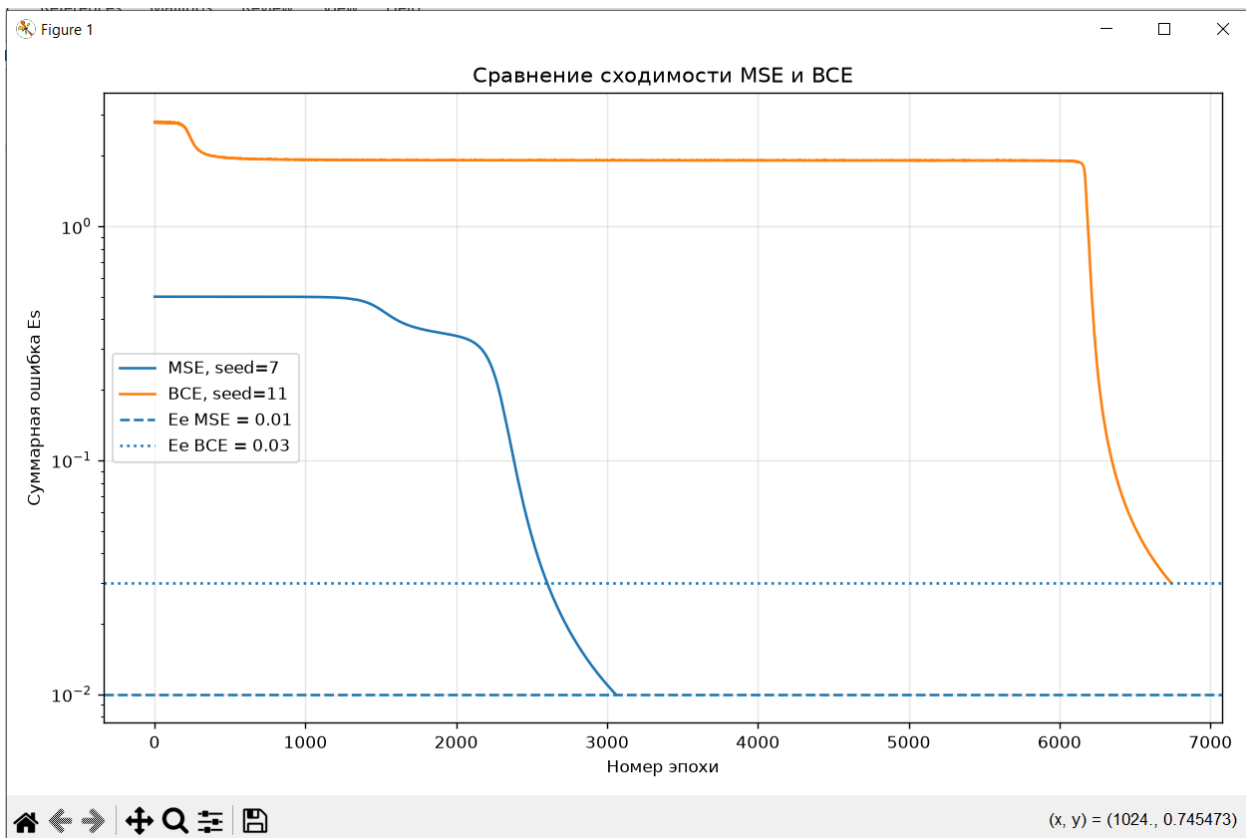
plt.show()

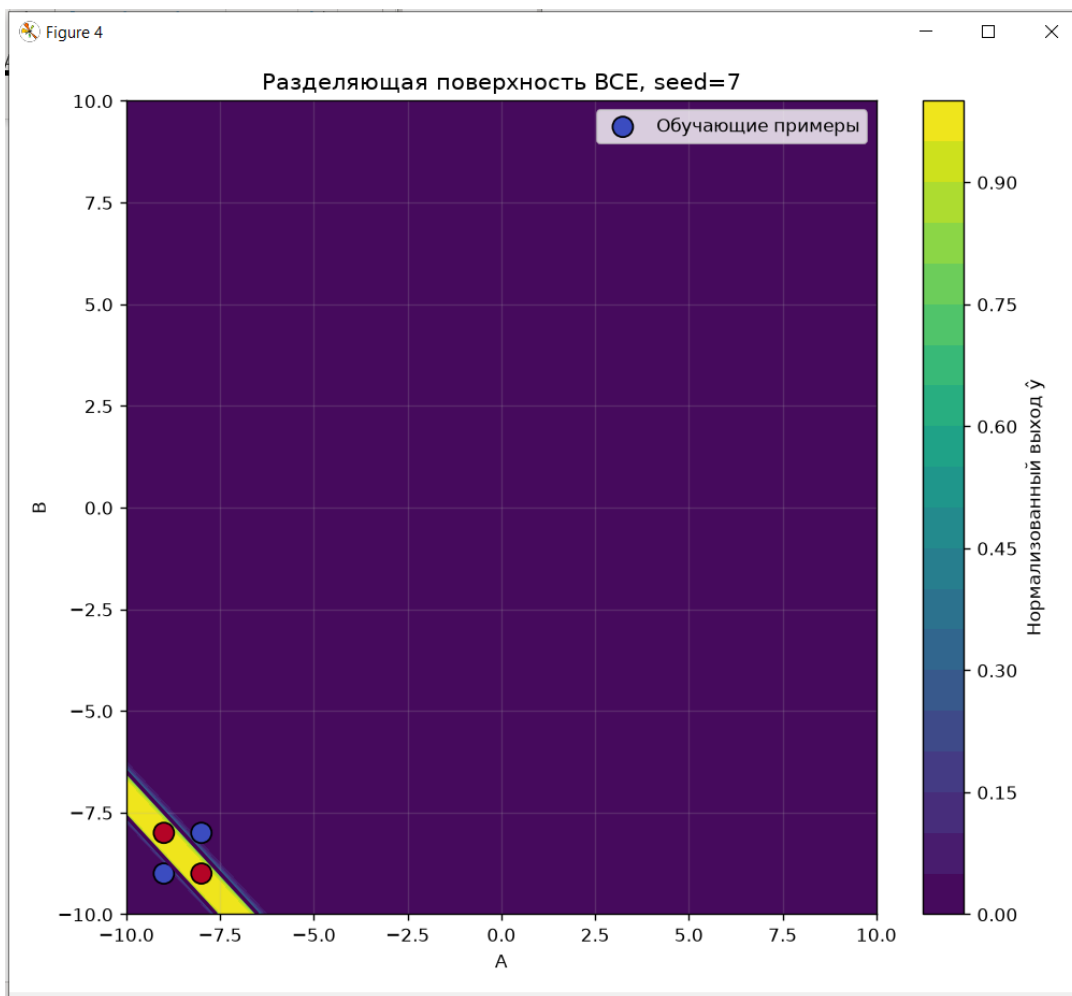
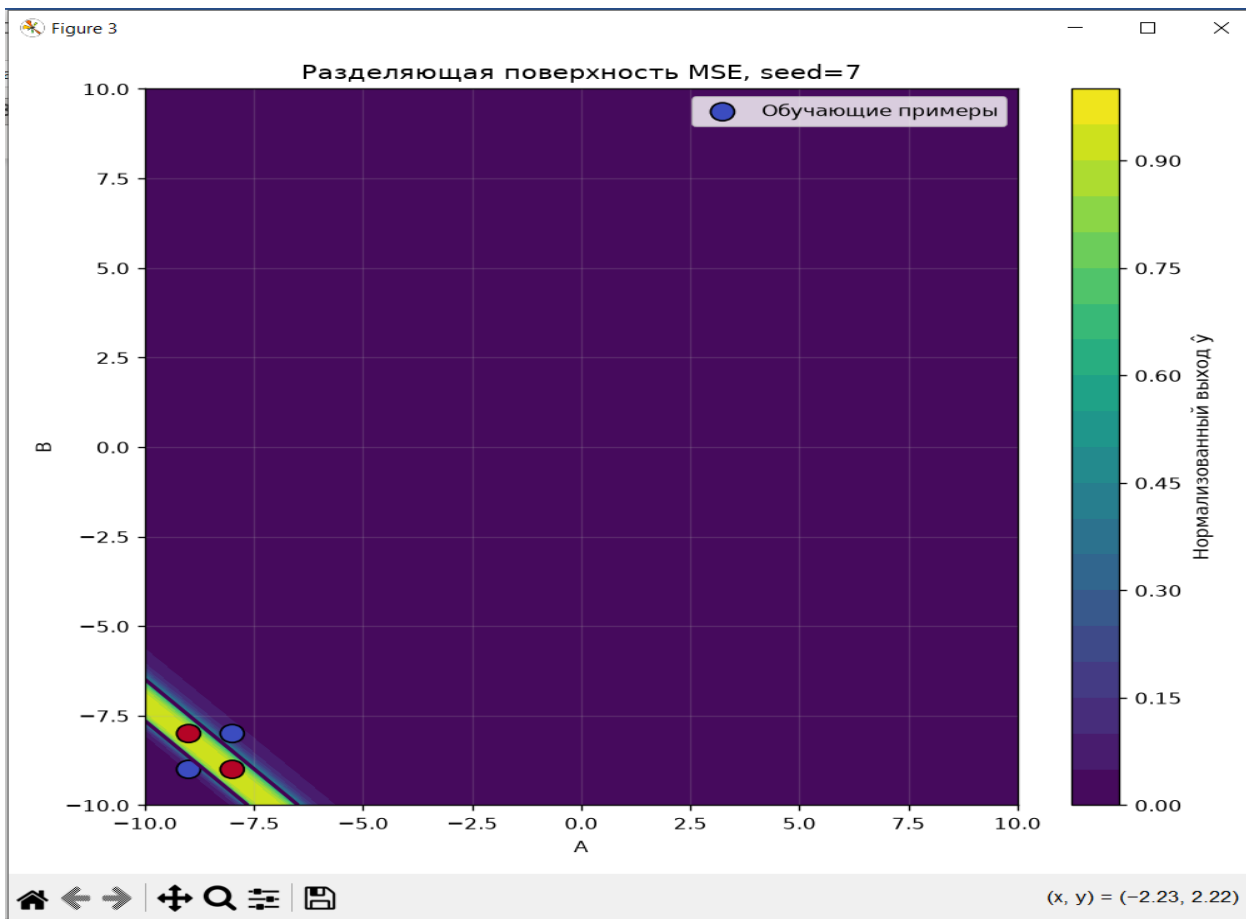
# После закрытия окон графиков
# можно вводить собственные A и B.
interactive_mode(
    bce_rep["model"]
)

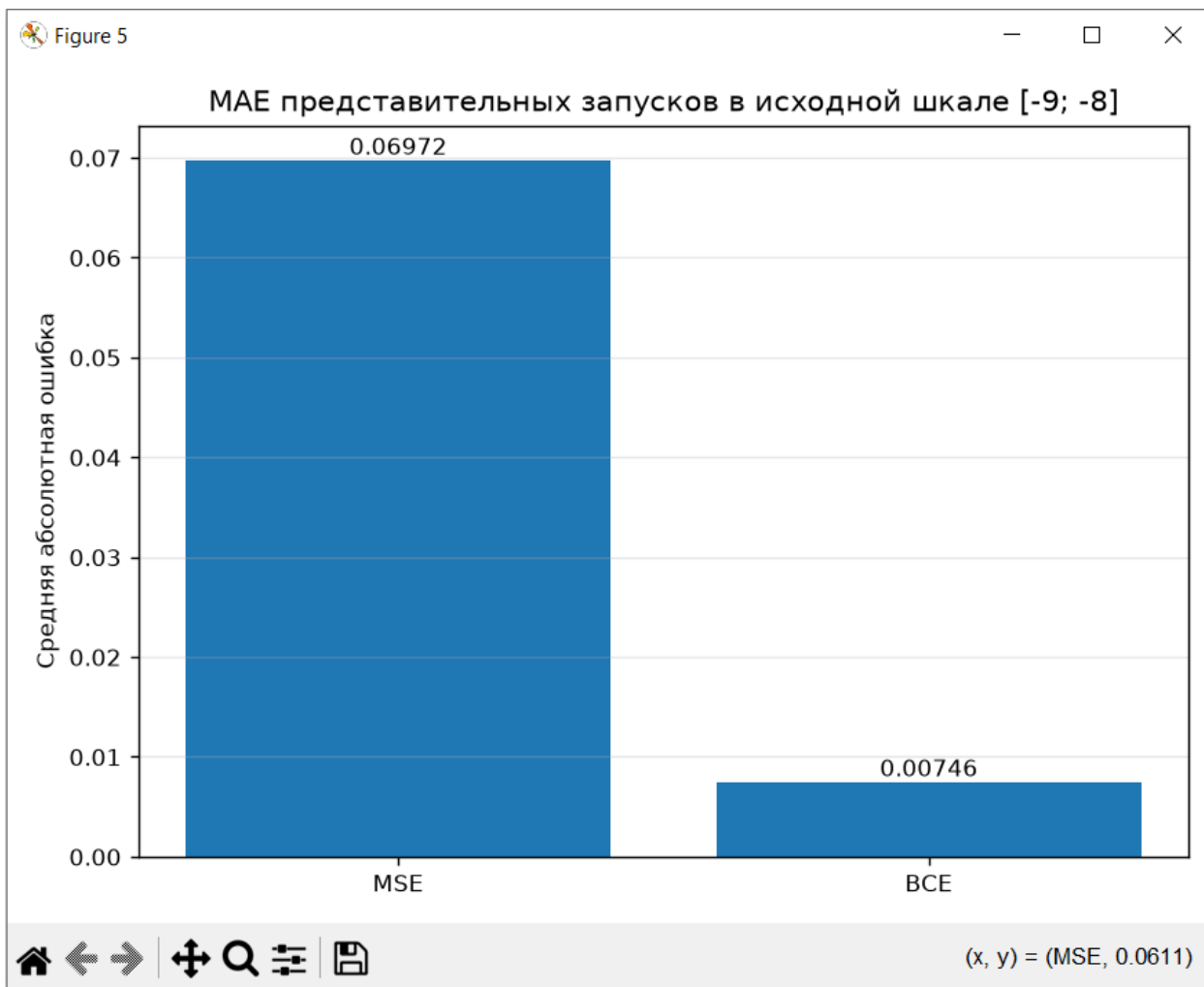
if __name__ == "__main__":
    main()

```

**Результат программы:**







## ЛАБОРАТОРНАЯ РАБОТА №2 — ВАРИАНТ 3

$c_0 = -9$ ,  $c_1 = -8$

### Обучение конфигурации A: MSE

seed= 7 | эпох= 3057 |  $E_s=0.009992$  | accuracy=1.00 | MAE=0.069673 | сходимость=ДА

seed= 8 | эпох= 3994 |  $E_s=0.009991$  | accuracy=1.00 | MAE=0.069382 | сходимость=ДА

seed= 9 | эпох=20000 |  $E_s=0.333934$  | accuracy=0.75 | MAE=0.336033 | сходимость=НЕТ

seed=10 | эпох=20000 |  $E_s=0.333819$  | accuracy=0.75 | MAE=0.335777 | сходимость=НЕТ

seed=11 | эпох= 2839 |  $E_s=0.009991$  | accuracy=1.00 | MAE=0.069748 | сходимость=ДА

## Обучение конфигурации В: BCE

seed= 7 | эпох=20000 | Es=1.910305 | accuracy=0.75 | MAE=0.331662 | сходимость=НЕТ  
seed= 8 | эпох=20000 | Es=1.910313 | accuracy=0.75 | MAE=0.331663 | сходимость=НЕТ  
seed= 9 | эпох=20000 | Es=1.910479 | accuracy=0.75 | MAE=0.331678 | сходимость=НЕТ  
seed=10 | эпох=20000 | Es=1.910351 | accuracy=0.75 | MAE=0.331668 | сходимость=НЕТ  
seed=11 | эпох= 6739 | Es=0.029965 | accuracy=1.00 | MAE=0.007459 | сходимость=ДА

=====

## СВОДКА ПО 5 ЗАПУСКАМ

=====

### MSE:

Достигли критерия: 3 из 5

Минимум эпох: 2839

Максимум эпох: 3994

Среднее эпох: 3296.67

Разброс: 1155

Средняя accuracy: 0.9000

Средняя MAE: 0.176123

### ВСЕ:

Достигли критерия: 1 из 5

Минимум эпох: 6739

Максимум эпох: 6739

Среднее эпох: 6739.00

Разброс: 0

Средняя accuracy: 0.8000

Средняя MAE: 0.266826

Представительный запуск MSE

| A    | B    | Цель | $\hat{y}$ norm | $\hat{y}$ real | Класс |
|------|------|------|----------------|----------------|-------|
| -9.0 | -9.0 | -9.0 | 0.0561         | -8.9439        | -9.0  |
| -9.0 | -8.0 | -8.0 | 0.9331         | -8.0669        | -8.0  |
| -8.0 | -9.0 | -8.0 | 0.9332         | -8.0668        | -8.0  |
| -8.0 | -8.0 | -9.0 | 0.0889         | -8.9111        | -9.0  |

Представительный запуск BCE

| A    | B    | Цель | $\hat{y}$ norm | $\hat{y}$ real | Класс |
|------|------|------|----------------|----------------|-------|
| -9.0 | -9.0 | -9.0 | 0.0046         | -8.9954        | -9.0  |
| -9.0 | -8.0 | -8.0 | 0.9935         | -8.0065        | -8.0  |
| -8.0 | -9.0 | -8.0 | 0.9935         | -8.0065        | -8.0  |
| -8.0 | -8.0 | -9.0 | 0.0123         | -8.9877        | -9.0  |

=====

РЕЖИМ ФУНКЦИОНИРОВАНИЯ — BCE

=====

A= -9.00, B= -9.00 |  $\hat{y}$ =0.0046 |  $y_{real}$ =-8.9954 | ближайший класс=-9.0  
A= -9.00, B= -8.00 |  $\hat{y}$ =0.9935 |  $y_{real}$ =-8.0065 | ближайший класс=-8.0  
A= -8.00, B= -9.00 |  $\hat{y}$ =0.9935 |  $y_{real}$ =-8.0065 | ближайший класс=-8.0  
A= -8.00, B= -8.00 |  $\hat{y}$ =0.0123 |  $y_{real}$ =-8.9877 | ближайший класс=-9.0  
A= -8.50, B= -8.50 |  $\hat{y}$ =0.9935 |  $y_{real}$ =-8.0065 | ближайший класс=-8.0  
A= -9.50, B= -8.20 |  $\hat{y}$ =0.5408 |  $y_{real}$ =-8.4592 | ближайший класс=-8.0

$A = -7.50$ ,  $B = -9.20$  |  $\hat{y} = 0.9323$  |  $y_{\text{real}} = -8.0677$  | ближайший класс = -8.0

**1. Сравнение сходимости MSE и BCE.** На графике видно, что в выбранном успешном запуске MSE достигла порога  $\epsilon_e = 0.01$  примерно за **3057 эпох**, тогда как BCE достигла своего порога  $\epsilon_e = 0.03$  только за **6739 эпох**. При этом обе функции сначала длительное время находятся на участке медленного изменения ошибки, после чего происходит резкое улучшение обучения. В данном эксперименте MSE сошлась быстрее, чем BCE. Непосредственно сравнивать абсолютные значения ошибок MSE и BCE между собой нельзя, поскольку это разные функции потерь и у них разные масштабы.

**2. Устойчивость к случайной инициализации.** Результаты пяти запусков показывают заметную зависимость обучения от начальных весов. Для MSE критерий остановки был достигнут в **3 случаях из 5**: при  $\text{seed} = 7, 8, 11$ . Число эпох успешных запусков составило от **2839 до 3994**, среднее — **3296,67 эпох**, разброс — **1155 эпох**. При  $\text{seed} = 9$  и 10 сеть остановилась на максимальных 20 000 эпохах с точностью 75%.

Для BCE ситуация оказалась менее устойчивой: только **1 запуск из 5**, при  $\text{seed} = 11$ , достиг заданного критерия. Остальные четыре запуска дошли до лимита 20 000 эпох и остановились с  $\text{accuracy} = 0.75$ . Таким образом, конкретно при выбранных параметрах обучения **MSE оказалась устойчивее к случайной инициализации**, несмотря на известное теоретическое преимущество BCE для бинарной классификации. Это не противоречие: теоретическое преимущество BCE не гарантирует, что при любых начальных весах и гиперпараметрах маленькая нелинейная сеть обязательно будет сходиться быстрее.

**3. Разделяющая поверхность MSE.** После успешного обучения сеть правильно разделила все четыре точки XOR. На графике область высокого значения выхода имеет форму диагональной полосы около обучающих точек. В неё попадают комбинации  $(-9, -8)$  и  $(-8, -9)$ , соответствующие классу  $c_1 = -8$ , а точки  $(-9, -9)$  и  $(-8, -8)$  располагаются за её пределами и относятся к  $c_0 = -9$ . Полученная область фактически формируется несколькими границами скрытого слоя, что подтверждает необходимость многослойной архитектуры.

**4. Разделяющая поверхность BCE.** Картина очень похожа на MSE: сеть также создала диагональную область для двух комбинаций класса  $c_1$ . При BCE переход между областями визуально получается достаточно резким. Это соответствует результатам выходного слоя: для четырёх обучающих примеров BCE получила значения 0.0046, 0.9935, 0.9935 и 0.0123, то есть ответы находятся очень близко к идеальным значениям 0 и 1.

Большая часть пространства  $[-10; 10] \times [-10; 10]$  на обоих графиках имеет практически нулевой выход. Это нормально интерпретировать осторожно: сеть обучалась только на четырёх точках около -9 и -8, поэтому достоверно судить о её поведении далеко за пределами обучающей области нельзя. Методичка требует построить поверхность во всём диапазоне  $[-10; 10]$ , но обучающая информация находится лишь в небольшом его фрагменте. PSAL 2.2

**Вывод:** В ходе лабораторной работы был исследован многослойный перцептрон архитектуры 2–2–1 для решения задачи XOR варианта 3 с числовыми классами  $c_0 = -9$  и  $c_1 = -8$ . Были исследованы функции потерь MSE и Binary Cross-Entropy, а также влияние случайной инициализации весов на процесс обучения.

Эксперимент показал, что обе функции потерь способны обеспечить правильное решение XOR, однако результаты существенно зависят от начальной инициализации весов. Конфигурация с MSE достигла критерия остановки в 3 из 5 запусков, тогда как BCE — только в 1 из 5. Для успешных запусков MSE среднее число эпох составило 3296,67, а успешному запуску BCE потребовалось 6739 эпох. Таким образом, при используемых параметрах MSE оказалась быстрее и устойчивее.

При этом после успешной сходимости BCE обеспечила более высокую точность восстановления исходных числовых значений. Средняя абсолютная ошибка составила 0,007459 для BCE и 0,069673 для MSE. Выходы BCE для обучающих примеров находились практически у идеальных значений 0 и 1, что после обратного преобразования дало результаты, близкие к  $-9$  и  $-8$ .

Визуализация разделяющих поверхностей показала, что обе конфигурации формируют нелинейную область, позволяющую корректно отделить две XOR-комбинации одного класса от двух комбинаций другого класса. Это подтверждает принципиальную линейную неразделимость XOR и необходимость наличия скрытого слоя. Выбор функции потерь влияет на процесс обучения, скорость сходимости и точность результата, однако не отменяет необходимости многослойной архитектуры.

В итоге для данного набора начальных весов MSE продемонстрировала большую устойчивость, тогда как BCE при успешной сходимости дала существенно более точный и уверенный результат. Следовательно, теоретическое преимущество BCE для бинарной классификации проявилось в качестве итогового предсказания, но не проявилось в большей устойчивости или скорости обучения при выбранных параметрах эксперимента.